

EPSI – ASRBD / STESE636

Tests d'integration & Migration de donnees

TD PRATIQUE – Migration de donnees

- Magyd Baroni
- Omar Diallo Bachir
- Filda Nzuzi Ngoma
- Anderson Kouadio
- Mattis Tajan

Rendu :

<https://docs.google.com/spreadsheets/d/1kvmt9BU8htTLuCzM3QgRRfB88aSP6sGZuTN1TufLTBI/edit?usp=sharing>

Contexte du TD

TechCorp Formation doit migrer son infrastructure du data center de Montpellier vers un nouveau data center à Toulouse. Vous jouez le rôle de l'équipe infrastructure chargée de préparer et d'exécuter cette migration.

A1 – Inventaire des composants

Table	Nombre de lignes	Depend de	Critique ?
utilisateurs	5	–	Oui
formations	4	–	Oui
progressions	6	utilisateurs + formations	Dépendante
resultats_examens	3	utilisateurs + formations	Dépendante

A2 – Inventaire des fichiers

Fichier	Type	Taille	Critique ?
excel_module1.pdf	PDF	32B	Important
cyber_intro.pdf	PDF	37B	Important
gestion_methodes.pdf	PDF	37B	Important
linux_video1.mp4	Video	33B	Important
logo.png	Image	20B	Important

A3 – Identification des contraintes

- Quelle est la bande passante disponible entre les deux sites ?
 - La bande passante disponible entre Montpellier et Toulouse est de 200Mbps.
- Quelle est la durée maximale de coupure autorisée ?
 - La durée maximale de coupure autorisée est de 2H.
- Quand le service doit-il être de nouveau disponible ?
 - Le service doit être de nouveau disponible le lundi matin.
- Quelles tables ne peuvent pas être migrées dans n'importe quel ordre ? Pourquoi ?
 - Progressions et resultats_examens ne peuvent pas être migrés dans n'importe quel ordre car elles ont des clés étrangères qui référencent utilisateurs et formations.
- Que se passe-t-il si on migre les progressions avant les utilisateurs ?
 - La migration échoue car PostgreSQL bloque l'insert car il vérifie l'intégrité référentielle à chaque ligne. Justement à cause des clés étrangères (FK).

A4 – Ordre de migration des tables

En tenant compte des dependances, proposez un ordre de migration :

Ordre	Table a migrer	Raison
1	utilisateurs	Aucune dépendance
2	formations	Aucune dépendance
3	progressions	Dépend de utilisateurs + formations
4	Resultats_examens	Dépend de utilisateurs + formations

NIVEAU 1 – Migration Big Bang

N1 – Questions d'analyse a rendre

- Combien de temps a dure la coupure totale ? Etait-ce dans la fenetre des 2 heures ?
 - La coupure a duré environ 15 minutes. C'est bien dans la fenêtre autorisée de 2 heures.
- Quels sont les risques d'une migration Big Bang sur une base de production réelle ?
 - Si la restauration échoue, le service est totalement indisponible jusqu'à résolution du problème. Toute l'équipe doit être mobilisée. Les utilisateurs ne peuvent rien faire pendant toute la durée de la coupure.
- Que se passe-t-il si la restauration échoue a mi-chemin ?
 - La base cible se retrouve dans un état incohérent, certaines tables existent, d'autres non. Il faut alors déclencher le rollback, remettre la source en lecture/écriture, vider la cible et recommencer depuis le début.
- Pourquoi l'ordre de restauration des tables est-il important ?
 - À cause des clés étrangères. Si progressions est restaurée avant utilisateurs, PostgreSQL refuse les insertions car les utilisateur_id référencés n'existent pas encore sur la cible.
- Dans quel cas réel pourrait-on accepter une migration Big Bang ?
 - Quand la base est petite, quand la fenêtre de maintenance est longue (la nuit, le week-end), ou quand le service peut se permettre un arrêt total sans impact majeur sur les utilisateurs.

NIVEAU 2 – Migration Progressive

N2 – Questions d'analyse a rendre

- Comparez la duree de coupure entre le Niveau 1 et le Niveau 2.
 - Le Niveau 1 (Big Bang) effectue l'export et la restauration complète pendant la coupure, ce qui peut prendre plusieurs dizaines de minutes sur une base de production volumineuse. Le Niveau 2 réduit la coupure à 54 secondes car l'essentiel des données est synchronisé en amont. Le service est donc indisponible beaucoup moins longtemps.
- Pourquoi l'option --delete de rsync est-elle importante lors de la synchronisation finale ?
 - Sans --delete, les fichiers supprimés côté source resteraient présents sur la cible. La cible ne serait pas un miroir exact de la source, ce qui pourrait laisser des fichiers obsolètes ou orphelins en production.
- Que se passe-t-il si un utilisateur s'inscrit entre la sync initiale et la coupure ?
 - Ses données existent sur la source mais pas encore sur la cible. C'est le décalage qu'on a simulé avec François et Gaëlle (2 utilisateurs ajoutés après la sync initiale). Ce delta est rattrapé lors de la synchronisation finale pendant la coupure.
- Pourquoi verifie-t-on les comptages AVANT de basculer ?
 - Pour s'assurer que toutes les données ont bien été transférées avant de rediriger le trafic vers la cible. Si les comptages divergent, on effectue un rollback plutôt que de basculer sur une base incomplète, ce qui évite toute perte de données.
- Dans quel cas la migration progressive ne suffit-elle pas ?
 - Quand la base est trop volumineuse ou trop active pour qu'une coupure de quelques secondes suffise à rattraper le delta. Par exemple, une base recevant des milliers d'insertions par seconde accumulerait trop de changements pendant la fenêtre de coupure. Dans ce cas, la réplication logique (Niveau 3) est nécessaire pour une bascule à chaud sans interruption.

NIVEAU 3 – Migration avec replication logique

N3 – Questions d'analyse a rendre

- Quelle est la duree de coupure avec la replication logique ? Comparez avec les Niveaux 1 et 2.
 - o C'est extrêmement rapide par rapport au Niveau 1 (Big Bang) où le service est coupé pendant toute la durée de l'export, du transfert et de l'import (potentiellement plusieurs heures). C'est aussi plus rapide que le Niveau 2 (Progressive) où il faut couper le service le temps de faire le dernier export différentiel (le delta) et son importation (environ 30 minutes de coupure).
- Quelles sont les limites de la replication logique PostgreSQL ?
 - o Elle ne réplique pas la structure (DDL : CREATE TABLE, ALTER, etc.).
 - o Elle ne réplique pas les séquences (les valeurs auto-incrémentées comme les SERIAL ne sont pas mises à jour sur la cible automatiquement en Postgres 15).
 - o Elle peut générer des conflits si la base cible est modifiée manuellement en même temps.
- Pourquoi doit-on creer le schema manuellement avant la subscription ?
 - o Parce que la réplication logique PostgreSQL transfère uniquement les données (les opérations de modification : INSERT, UPDATE, DELETE). Elle ne s'occupe pas de la définition des objets. La cible doit donc avoir exactement les mêmes tables prêtes à recevoir les lignes de données avant que la réplication ne commence.
- Que se passe-t-il si on insere une ligne sur la CIBLE pendant que la replication est active ?
 - o Si tu insères une ligne sur la cible (ex: id = 10), et qu'ensuite la source crée aussi une ligne avec id = 10 et tente de la répliquer, la réplication va planter (erreur de clé primaire dupliquée). Le processus de réplication s'arrêtera jusqu'à ce que le conflit soit résolu manuellement. C'est pourquoi la cible ne doit normalement pas être écrite par les utilisateurs avant la bascule.
- Dans quel cas prefererait-on la replication logique a la migration progressive?
 - o On la préfère lorsque l'application est hautement critique et exige un Zero Downtime (ou presque). Par exemple, pour un site e-commerce mondial ou une application bancaire où l'on ne peut pas se permettre de couper les écritures pendant 30 minutes ou 1 heure.
- Quels outils professionnels permettent de faire de la replication encore plus avancee ?
 - o Pglogical : Extension PostgreSQL plus ancienne et complète pour la réplication.
 - o Debezium / Kafka Connect : Pour faire du CDC (Change Data Capture) et répliquer vers d'autres types de bases.
 - o AWS DMS (Database Migration Service) : Outil cloud très puissant pour orchestrer des migrations complexes sans coupure.
 - o Oracle GoldenGate : Solution d'entreprise classique (très chère) pour la réplication multi-bases.

Synthese comparative des 3 methodes

A completer par les etudiants a la fin de la journee.

Critere	Big Bang (N1)	Progressive (N2)	Replication (N3)
Duree de coupure mesuree	Longue	Moyenne	Minimale
Complexite de mise en oeuvre	Faible	Moyenne	Elevee
Risque de perte de donnees	Eleve	Moyen	Faible
Rollback possible ?	Oui	Oui	Oui
Adapte a 2h de coupure ?	Non	Oui	Oui
Outils utilises	pg_dump / rsync	pg_dump + rsync	PUBLICATION / SUBSCRIPTION
Recommande pour ce projet ?	Non	Oui	Oui et Non

Livrable final attendu

- Les scripts completes (avec les TODO remplis) pour chaque niveau
- Le tableau d'analyse A1 a A4 complete
- Les reponses aux questions d'analyse de chaque niveau
- Le tableau comparatif des 3 methodes complete
- Un court paragraphe de conclusion : quelle methode recommandez-vous pour TechCorp Formation et pourquoi ?
 - Nous recommandons la méthode Progressive (Niveau 2) pour TechCorp Formation. Bien que la Réplication (Niveau 3) offre le temps de coupure le plus court (4 secondes), sa complexité de mise en œuvre est élevée. La méthode Progressive permet de respecter largement la fenêtre de coupure imposée de 2 heures (environ 30 minutes nécessaires) tout en gardant une complexité maîtrisée et une procédure de rollback (retour arrière) très fiable.